Introduction of Stack

A stack is a linear data structure commonly used in programming. It is a sequential collection of elements where insertion and deletion operations are performed only at one end, called the top of the stack.

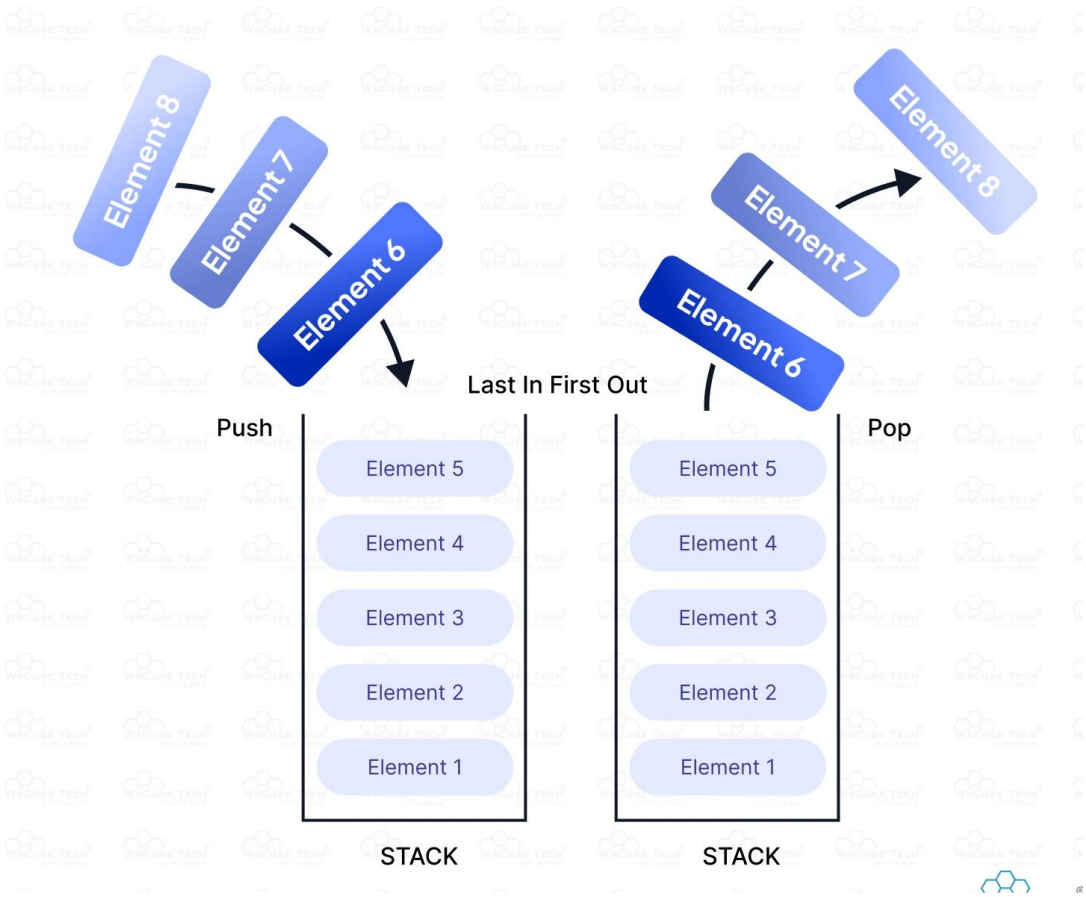
Stacks operate on the principle of LIFO (Last-In, First-Out) or FILO (First-In, Last-Out). A stack uses a variable called top, which acts as a pointer that indicates the position of the most recently inserted element in the stack. Examples: pile of books, stack of plates in restaurant, rings in a box, disk on the top of another, etc.

Key Concepts

LIFO: LIFO stands for Last-In, First-Out. Whenever a new element is inserted last, it will be removed or deleted first. This is called LIFO.

FILO: FILO stands for First-In, Last-Out. Whenever a new element is inserted first, it will be removed last. This is called FILO. In stacks, FILO and LIFO are essentially the same concept. LIFO is more commonly used terminology.

Top: Top is a pointer or variable that tracks the most recently added element, which is at the topmost position of the stack. All insertion and deletion operations are performed at the top. The top also helps to check the current position of the stack and identify where the latest element is stored.

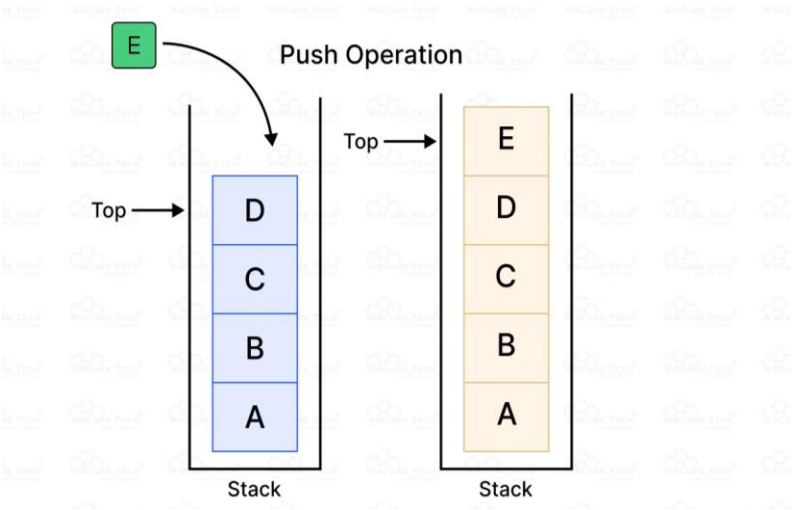


Representation of Stack Data Structure

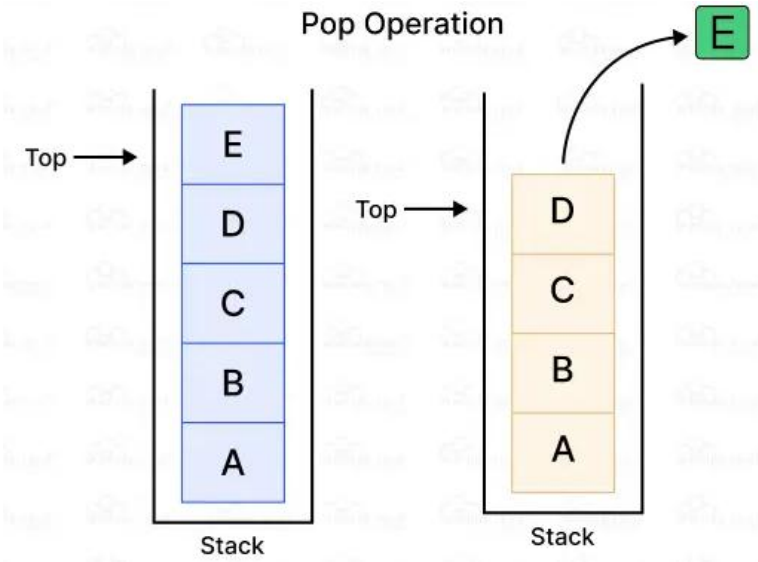
Types of Stack Operations / Stack as an ADT

There are several operations that can be performed on a stack. They are:

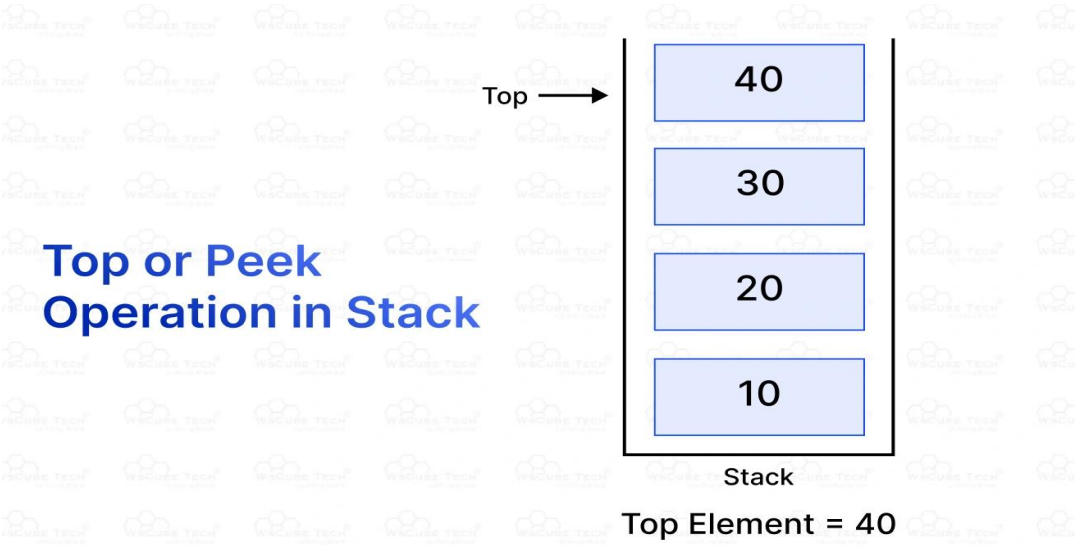
1. **push( ):** When a new element is inserted at the top of the stack, such an operation is called a push( ) operation. In stack implementation using an array, while inserting or adding a new element onto the top of the stack, the top is incremented by 1. During the push operation, an array-based stack must be checked to see if the stack is full before adding a new element. If the stack is already full and we try to push a new element, then stack overflow occurs, and the push operation will not proceed further.



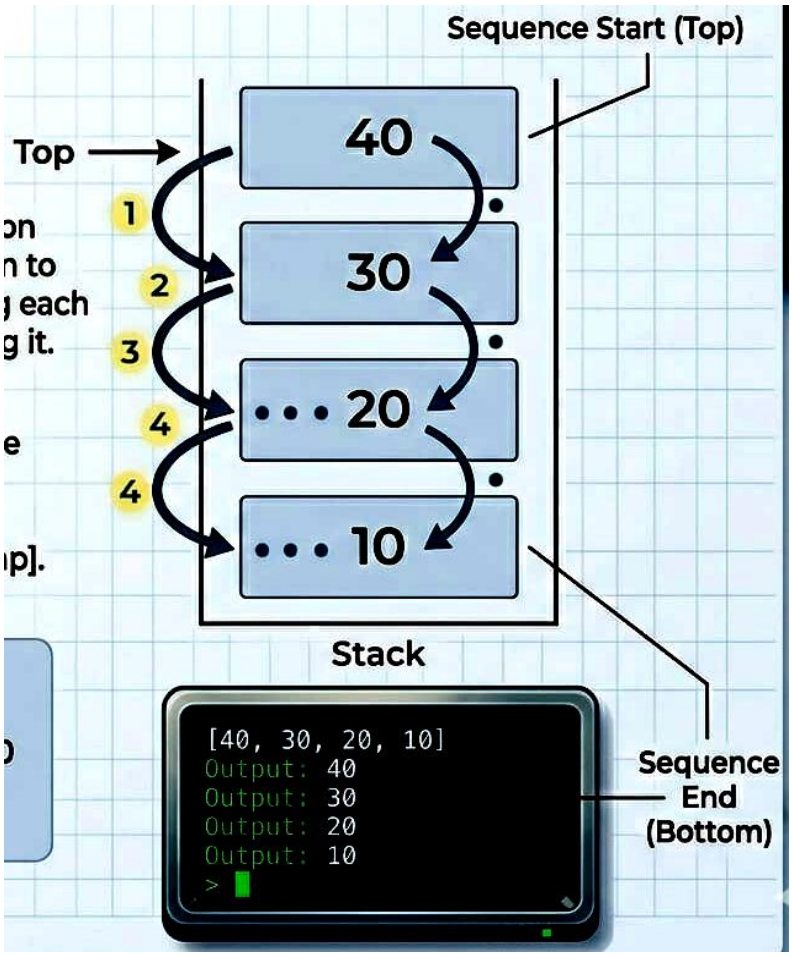
2. **pop( ):** When an element is removed from the top of the stack, such an operation is called a pop( ) operation. While popping an element from the stack, the top is immediately decremented by 1, pointing to the current position of the topmost element that remains in the stack. We do not manually or physically delete the element from the stack, but it is logically removed and hence no longer accessible. If a new element is later pushed onto the stack, the top is incremented by 1, and the new element overwrites the previously removed element automatically.



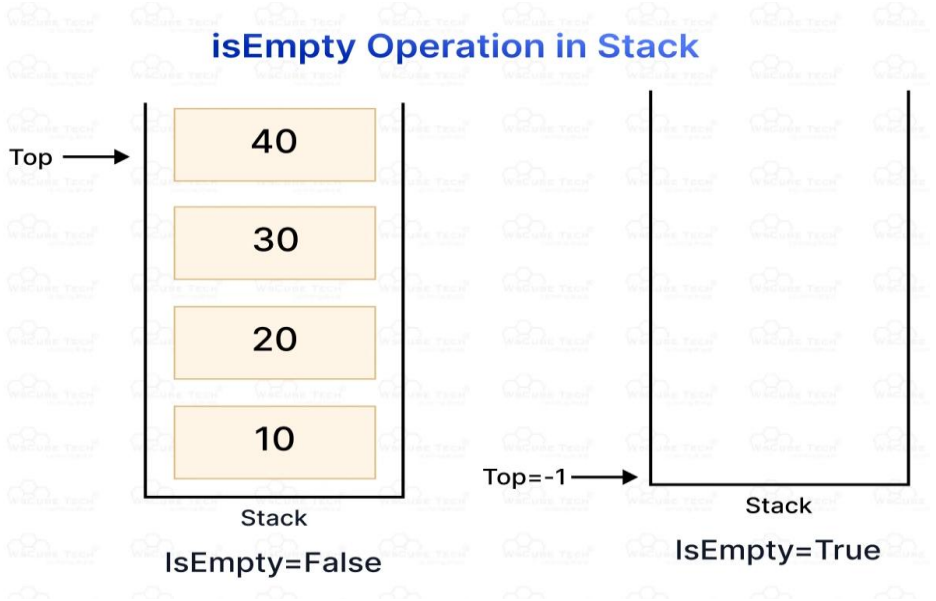
3. **peek( )/ top( ):** The peek( ) operation returns the element that was inserted last, i.e., the topmost element of the stack. In other words, it fetches or shows the last added element or the element present at the top without removing it from the stack.



4. **display( ):** The display( ) operation prints or displays all the elements of the stack in reverse order, i.e., from the top to the bottom, with respect to the order in which they were pushed.

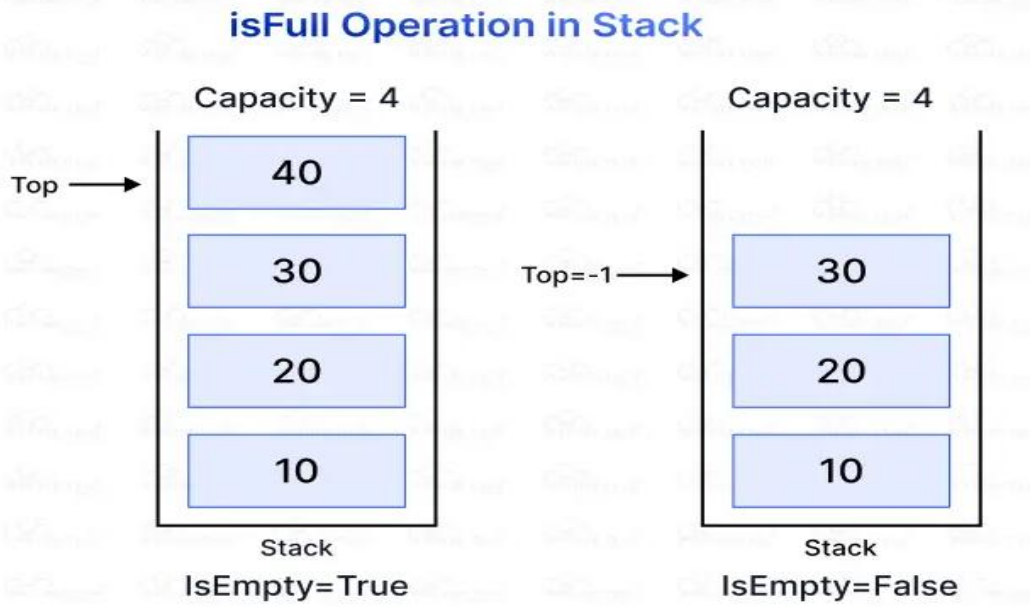


5. **isEmpty( )/ Undeflow:** The isEmpty( ) operation is used to check whether the stack contains any elements or not (simply, whether it is empty or not). It returns a boolean value, i.e., true (1) or false (0). If the stack is considered to be empty, it returns true, otherwise, it returns false.



The primary purpose of the isEmpty( ) operation is to prevent errors such as stack underflow, which occurs when trying to perform operations like pop( ), peek( ), or display( ) on a stack that has no elements. Programmers typically use isEmpty( ) as a precautionary check before attempting these other operations.

6. **isFull( ) / Overflow:** The isFull( ) operation checks whether the stack has reached its maximum capacity. Since a stack implemented using an array has a fixed and predefined size, its capacity cannot be changed during the runtime of the program.



The primary purpose of isFull( ) is to prevent a stack overflow error, which occurs when trying to perform the push( ) operation on a stack that has already reached its maximum capacity or does not have enough space. If the stack is full and cannot accept more elements, it returns true, otherwise, it returns false if the stack still has room for additional elements.

Implementation of Stack

A stack can be implemented in the following two ways:

1. Array-Based Implementation of Stack (Static Implementation)
2. Linked List-Based Implementation of Stack (Dynamic Implementation)

Array-Based Implementation of Stack

To implement a stack, we use the concept of an array. The properties of the array become the properties of the stack as well. In an array-based stack, a **fixed size** is defined. An index pointer named top is used to keep track of the index of the topmost element. Initially, top = -1 to indicate that the stack is empty (no elements are present). When there is space in the stack, top is first incremented by 1, and then the new element is inserted at stack[top].

Algorithm for push( ) operation:

```
push(item)
  if top == MAX_SIZE - 1
    print "Stack Overflow"
  else
    set top ← top + 1
    set stack[top] ← item
```

Algorithm for pop( ) operation:

```
pop()
  if top == -1
    print "Stack Underflow"
  else
    set temp ← stack[top]
    set top ← top - 1
    return temp
```

Algorithm for peek( ) operation:

```
peek()
    if top == -1
        print "Stack Underflow"
    else
        return stack[top]
```

Algorithm for isEmpty( ) operation:

```
isEmpty()
    if top == -1
        return 1      // stack is empty
    else
        return 0      // stack is not empty
```

Algorithm for isFull( ) operation:

```
isFull()
    if top == MAX_SIZE - 1
        return 1      // stack is full
    else
        return 0      // stack is not full
```

Applications of Stack

A stack is widely used in computer science for various purposes. Some of the important applications of stack are:

1. Memory Management
2. Function Calling (Call Stack)
3. Undo–Redo Functionality in Editors
4. Balanced Parentheses Checking
5. Expression Evaluation and Conversion
6. Backtracking

7. Reversing a String
8. Recursive Call
9. Browser History Management
10. Task Management

Analysis of Algorithm:

Operation	Time Complexity	Space Complexity
push( )	O(1)	O(1)
pop( )	O(1)	O(1)
peek( )	O(1)	O(1)
isFull( )	O(1)	O(1)
isEmpty( )	O(1)	O(1)
display( )	O(n)	O(1)